

AI 求职智能匹配智能体 — 方案说明

一、问题诊断

学生求职面临两大痛点：(1) 岗位海量，逐一甄别与自身背景、能力、兴趣的匹配度耗时巨大；(2) 锁定心仪岗位后，无法判断简历与岗位的契合程度，也不清楚如何优化以通过简历初筛（ATS/HR 筛选）。本方案聚焦这两点，构建一个"先精准匹配、再针对性优化"的闭环智能体。

二、方案设计

产品形态为 Next.js 全栈 Web 应用，左侧功能区、右侧"简历优化前后对比"示例区，流程分四步：

1. 简历录入：支持上传 PDF / DOCX 文件自动抽取文本，或直接粘贴。
 2. 简历解析：大模型把简历抽取为结构化画像（技能、经历、教育、求职意向）。
 3. 岗位智能匹配：将画像与岗位库交给大模型做多维语义评估（技能契合、经历相关、意向匹配、经验要求），输出 Top 岗位、匹配度评分及匹配理由，解决"找岗难"。
 4. 简历诊断优化：针对用户选定岗位，对比简历与 JD，给出预估初筛命中率、命中/缺失关键词、可执行优化建议及按 STAR 法则的改写示例，直接提升"初筛命中率"。
- 岗位库覆盖学生高频领域（互联网研发、产品运营等），含应届校招与实习岗，字段对齐真实招聘平台 JD 结构。

三、AI 工具选型理由

- 大模型选 DeepSeek：OpenAI 兼容接口、国内访问稳定、成本低，在中文语义理解、结构化抽取与指令遵循上表现优秀，匹配理由与简历建议专业可读。
- 召回方案：采用"大模型直接对岗位库语义排序"而非向量检索。岗位库规模可控，单次调用即可完成召回+排序+解释，架构更简、无额外向量服务依赖，召回质量更优。此为关键工程权衡。
- 文件解析：PDF 用 pdfjs-dist、DOCX 用 mammoth 在服务端抽取文本，避免前端暴露逻辑。

四、关键配置

- 适配层抽象大模型接口，密钥仅经环境变量（DEEPSEEK_API_KEY）注入、不入代码库；前端不接触密钥。
- LLM 输出强约束为 JSON，并做容错提取（去除代码块包裹、截取首尾花括号），保证解析稳定。
- 匹配综合分由模型按多维度给出，前端按命中率分级着色展示，交互直观。

五、迭代记录

1. 搭建 Next.js + 简历解析/岗位匹配/简历优化三个 API。
2. 因本地 embedding 依赖需原生编译、网关又不支持 embedding，召回改为大模型直接语义排序。
3. 调用网关时遇 403：定位为网关 WAF 拦截 SDK 默认 User-Agent，改用纯 fetch 调用解决。
4. 大模型后端由 Claude 网关切换为 DeepSeek（OpenAI 兼容）。
5. 新增 PDF / DOCX 简历文件上传与文本抽取。
6. 部署遇阻：Vercel 在国内访问不稳定，改用腾讯云 EdgeOne Pages 部署。
7. 页面美化：左右分栏、蓝鲸主题背景、简历优化前后对比示例。

六、效果评估

以"Java 后端 2026 届"简历实测：匹配结果将"后端开发（校招）95 分""Java 后端实习 88 分"准确置顶，相关岗位（测试开发等）合理降权；针对后端岗的诊断准确识别出"微服务/分布式/高并发"等缺失关键词，并给出可量化的 STAR 改写示例（如"响应时间 300ms→50ms""支持 200+并发"）。匹配精准、建议可落地，验证了方案有效性。PDF/DOCX 上传、中英文简历抽取均经本地实测通过。